VLSI Design Automation (EECE 5186C/6086C)

Ranga Vemuri

Home Work – 1

In this homework, you will implement a 2D-placement algorithm. This is a group homework. Two students per group. You may choose your partner. If the sum of M numbers of the team members is odd, you will implement simulated annealing. Otherwise, you will implement force-directed placement.

You will be given netlists in the following Python dictionary format in a file (see Place_5 file):

```
data = {
    'grid_size': 3,

    'cells': {
        'CELL_0': {'type': 'MOVABLE', 'fixed': False, 'position': (1, 2)},
        'CELL_1': {'type': 'MOVABLE', 'fixed': False, 'position': (1, 0)},
        'CELL_2': {'type': 'MOVABLE', 'fixed': False, 'position': (0, 0)},
        'CELL_3': {'type': 'MOVABLE', 'fixed': False, 'position': (0, 1)},
        'IO_0': {'type': 'IO', 'fixed': True, 'position': (2, 1)},
    },

    'nets': [
        {'name': 'NET_0', 'cells': ('IO_0', 'CELL_2')},
        {'name': 'NET_1', 'cells': ('CELL_0', 'CELL_1')},
        {'name': 'NET_10', 'cells': ('CELL_1', 'IO_0')},
        {'name': 'NET_11', 'cells': ('CELL_1', 'CELL_2')},
        {'name': 'NET_12', 'cells': ('CELL_3', 'CELL_2')},
        {'name': 'NET_2', 'cells': ('CELL_0', 'CELL_1')},
        {'name': 'NET_3', 'cells': ('CELL_2', 'CELL_3')},
        {'name': 'NET_4', 'cells': ('CELL_3', 'CELL_2')},
        {'name': 'NET_5', 'cells': ('CELL_2', 'CELL_0')},
        {'name': 'NET_6', 'cells': ('CELL_0', 'CELL_3')},
        {'name': 'NET_7', 'cells': ('CELL_3', 'CELL_2')},
        {'name': 'NET_8', 'cells': ('IO_0', 'CELL_3')},
        {'name': 'NET_9', 'cells': ('CELL_0', 'IO_0')},
    ]
}
```

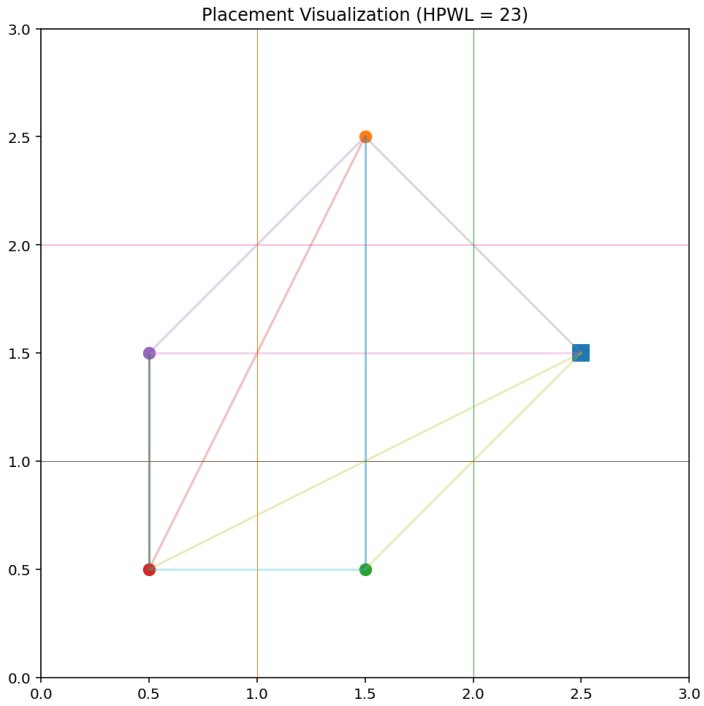
For cells, **fixed: True** flag indicates that the cell is locked in that position. False indicates that the cell's position is an initial random position and the cell is allowed to move. 'type' key is a placeholder and has no use in this homework but should be retained and printed out in the output file.

All nets are two terminal nets. There may be parallel nets.

Your goal is to find a placement such that the total HPWL is as small as possible.

You final placement should be printed in the same format as above except that the positions of all cells will be for your final placement. Note that the output file should contain the complete netlist information as above.

The Place_5 netlist is shown below pictorially along with some netlist data:



Grid Size = 3
Cells = 5
Fixed IO Cells = 1
Movable Cells = 5 – 1 = 4
Nets = 13 (nets may be parallel)
Total HPWL for the initial placement = 23

You will receive two sets of netlists:

- Ptest-Tests: For these netlists, we are aware of placements with known best HPWL values. (It is possible that there are even better placements. These are just the best ones known.) Netlist data along with these values are provided below:

Sl.	Netlist	Grid Size	Cells	Fixed I0 Cells	Nets	Initial HPWL	Best Known Placement's HPWL
1	Ptest_25	7	25	2	24	159	49
2	Ptest_50	9	50	5	49	403	121
3	Ptest_100	13	100	10	99	1281	256
4	Ptest_500	29	500	50	499	14,650	2,005
5	Ptest_1000	40	1,000	30	999	39,371	3,004
6	Ptest_5000	90	5,000	64	4,999	442,994	14,243
7	Ptest_10000	127	10,000	214	9,999	1,247,867	41,219
8	Ptest_15000	155	15,000	225	14,999	2293305	57,840
9	Ptest_20000	179	20,000	129	19,999	3,527,844	58,858
10	Ptest_25000	200	25,000	137	24,999	4,921,479	71,044

You can use Ptest netlists for testing and tuning your programs.

- Place-Benchmarks: For these netlists we do not know the best placements. The following data are provided for these netlists:

Sl.	Benchmark	Grid Size	Cells	Fixed I0 Cells	Nets	Initial HPWL
1	Place_25	7	25	2	51	242
2	Place_50	9	50	5	104	574
3	Place_100	13	100	10	238	2,154
4	Place_500	29	500	27	1,011	19,463
5	Place_1000	40	1,000	46	2,457	66,872
6	Place_5000	90	5,000	96	13,389	808,528
7	Place_10000	127	10,000	111	24,352	2,066,884
8	Place_15000	155	15,000	122	41,225	4,247,371
9	Place_20000	179	20,000	193	43,838	5,248,680
10	Place_25000	200	25,000	201	66,478	8,906,514

Your programs are expected to be in Python. You can develop your programs on any machine but the final results should be gathered on the VLSI Lab machines. Python3 and VSCode are available on these machines.

You need to evaluate your programs on both sets of netlists and report results. You should not allow more than one hour of CPU time for any netlist on the VLSI Lab machines.

Your submission must be a single .zip file and contain the items described below.

1. Your source code, properly commented and indented for easy readability and submitted as a single file. Name this <LastName1>_<LastName2>_SA.py or <LastName1>_<LastName2>_FD.py. Include execution instructions at the top.

2. Name the result files as <LastName1>_<LastName2>_<OriginalNetlistName>. Put these files in two separate folders and name the folders <LastName1>_<LastName2>_<OriginalFolderName>. Result files should be pretty printed for easy readability similar to the netlist files.

3. For random number generation, use a seed and hard-code the seed value. Use the same seed for all runs. Reproducibility of results is essential.

4. A report as a pdf file. It should contain:

    1. Student names and M numbers.
    2. Description of the algorithm, key implementation decision, tuning procedure used, and the final tuning parameter values.

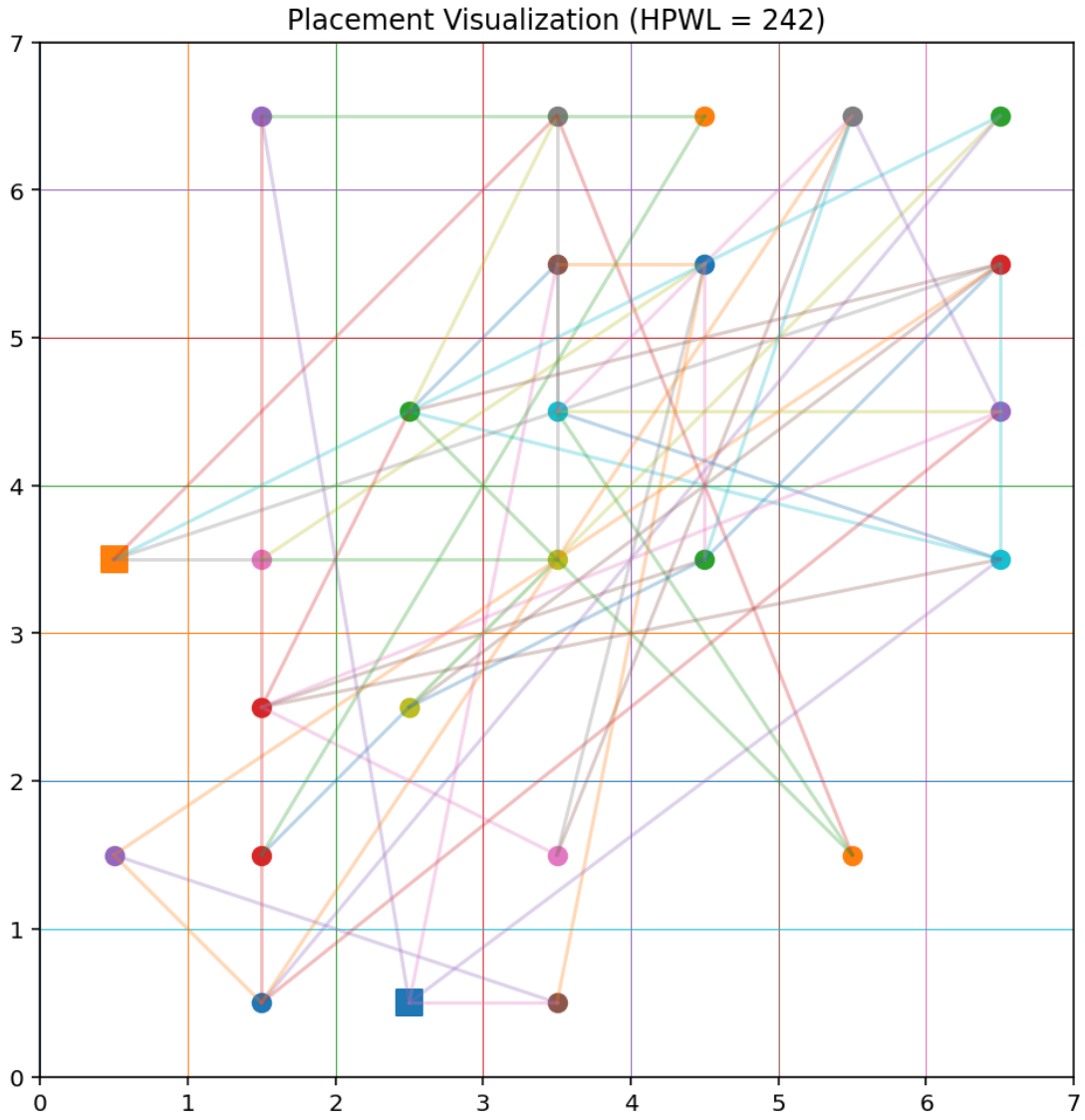
3. Result tables indicating the quality of the result, CPU and memory requirements.
4. For the SA, the four convergence plots discussed in the class for Place_500 and only the HPWL convergence plots for all the netlists.
5. For the FD, HPWL vs. iteration number and number of cells moved from the previous iteration vs. iteration number plots for all the netlists.
6. Any other information and experimental data you feel necessary to judge the quality of your algorithm, your software and your effort.
7. A retrospective describing what you think is the cause of the good/poor performance of your tool and how might you be able to improve the performance (both execution speed and quality of result) of your tool if you had more time?

Note on use of AI Tools for this homework only: You are allowed to use AI tools for coding (but not report writing) providing you describe which tool was used and how. You are required to share your chat interactions if asked. All other academic misconduct rules are in effect.

You may be asked to give a demo/presentation on your projects and required to answer any questions.

Appendix: More visualizations for your information.

Place_25:



Place_50: